



CELEBAL
TECHNOLOGIES

SUPERSTORE DATA PROCESSING USING PYSPARK

Celebal Summer Internship 2026

Week 5 Assignment

Data Cleaning, Transformation and Analysis using PySpark DataFrames

Dataset: Modified Superstore Dataset

Original Superstore dataset with intentionally introduced null values and duplicate records to demonstrate data cleaning techniques in PySpark.

Submission Date:

21-06-2026

Prepared By:

Keshav Kumar sharma

1. Objective

The objective of this assignment was to understand Apache Spark fundamentals and perform data cleaning, transformation, filtering, and aggregation operations using PySpark DataFrames.

2. Dataset Used

Dataset Name:

Superstore_Modified.csv

Note : For this assignment, I used the Superstore dataset. Since the original dataset had very few data quality issues, I modified it slightly by introducing some null values and duplicate records.

The purpose of doing this was to practice and demonstrate different data cleaning techniques in PySpark, such as handling missing values, removing duplicate records, changing data types, and performing data analysis.

After creating these issues, I applied various Spark DataFrame operations to clean and process the dataset and then generated business insights using filtering and aggregation functions.

3. Apache Spark Setup

The following environment was configured:

- Python
- PySpark 3.5.1
- Java JDK 17
- Jupyter Notebook

Spark session was successfully created.

Superstore Data Processing using PySpark

Objective: To understand Spark DataFrames and perform data cleaning, transformation and aggregation on Superstore dataset.

```
[36]: pip install pyspark

Requirement already satisfied: pyspark in c:\users\kesha\appdata\roaming\python\python313\site-packages (3.5.1)
Requirement already satisfied: py4j==0.10.9.7 in c:\users\kesha\appdata\roaming\python\python313\site-packages (from pyspark) (0.10.9.7)
Note: you may need to restart the kernel to use updated packages.
WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\kesha\anaconda3\Lib\site-packages)
WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\kesha\anaconda3\Lib\site-packages)
WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\kesha\anaconda3\Lib\site-packages)
```

```
[10]: import os

os.environ["SPARK_LOCAL_HOSTNAME"] = "localhost"

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("CEI_Week5") \
    .getOrCreate()

print("Spark Ready")

Spark Ready
```

```
[11]: from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("Superstore Analysis") \
    .getOrCreate()
```

4. Data Loading and Schema Validation

Dataset was loaded using Spark DataFrame.

```
[12]: #Load Dataset

df = spark.read.csv("Superstore_Modified.csv", header=True, inferSchema=True)
df.limit(6).toPandas()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	Postal Code	Region	Product ID	Category	Sub-Category	Product Name
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gite	Consumer	United States	Henderson	42420	South	FUR-BO-10001798	Furniture	Bookcases	Some Collec Book
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gite	Consumer	United States	Henderson	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon De Fa Upholstr Stad Chai
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	90036	West	OFF-LA-10000240	Office Supplies	Labels	Adhe Add Label Typewri
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	33311	South	FUR-TA-10000577	Furniture	Tables	Bret CR4 Series Rectang T
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon I 'N Roll Sys
5	6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	90032	West	FUR-FU-10001487	Furniture	Furnishings	El Express Wood Plastic C Acc

```
[15]: df.printSchema()

root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: string (nullable = true)
|-- Quantity: string (nullable = true)
|-- Discount: string (nullable = true)
|-- Profit: string (nullable = true)

Observation:

While inspecting the schema, Sales, Quantity and Discount were inferred as string instead of numeric types.

These columns need type conversion before performing aggregations and calculations.
```

5. Data Quality Assessment

Null Value Analysis

Missing values were intentionally introduced into the Sales column for learning and demonstration purposes.

Null values were identified using Spark functions.

```
[17]: #Checking Null values

from pyspark.sql.functions import *

df.select([
    count.when(col(c).isNull(), c).alias(c)
    for c in df.columns
]).toPandas()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Disc
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	10	0	

1 rows x 21 columns

Observation

The dataset contained missing values which required cleaning before analysis.

Duplicate Record Analysis

Duplicate rows were intentionally added to demonstrate duplicate detection and removal.

```
[21]: #Checking Duplicate Records

total_rows = df.count()

unique_rows = df.dropDuplicates().count()

print("Total Rows:", total_rows)
print("Unique Rows:", unique_rows)
print("Duplicate Rows:", total_rows - unique_rows)

Total Rows: 10004
Unique Rows: 9994
Duplicate Rows: 10
```

Observation

Duplicate records can impact analytical results and should be removed before processing.

6. Data Cleaning

Handling Missing Values

Null values were replaced using appropriate Spark DataFrame operations.

```
[18]: #Taking Average value

from pyspark.sql.functions import avg

df.select(avg("Sales")).show()

+-----+
|      avg(Sales)|
+-----+
|236.50581523289495|
+-----+

[19]: #filling null values

mean_sales = df.select(avg("Sales")).collect()[0][0]

df = df.fillna({"Sales": mean_sales})

[20]: #checking again

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).toPandas()

[20]:
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Observation

Missing values Sales were successfully handled.

Removing Duplicate Records

Duplicate records were removed using:

```
[22]: #Removing Duplicate Rows
df = df.dropDuplicates()

[23]: print("Total Rows after removing duplicates:", df.count())
Total Rows after removing duplicates: 9994
```

Observation

Dataset integrity improved after removing duplicate rows.

7. Data Type Transformation

The following columns were converted:

Column	Converted Type
--------	----------------

Sales	Double
-------	--------

Quantity	Integer
----------	---------

Discount	Double
----------	--------

```
[25]: #Schema Modification (Casting)

from pyspark.sql.types import DoubleType, IntegerType

df = df.withColumn(
    "Sales",
    col("Sales").cast(DoubleType())
)

df = df.withColumn(
    "Quantity",
    col("Quantity").cast(IntegerType())
)

df = df.withColumn(
    "Discount",
    col("Discount").cast(DoubleType())
)
```

```
[26]: #Verify Schema Again

df.printSchema()

root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Discount: double (nullable = true)
|-- Profit: string (nullable = true)
```

Observation

After converting the columns to numeric data types, aggregation and analytical operations could be performed accurately.

8. Data Analysis and Aggregation

Category Wise Sales

```
[28]: #Category Wise Sales

from pyspark.sql.functions import sum

df.groupBy("Category") \
  .agg(sum("Sales").alias("Total_Sales")) \
  .show()
```

Category	Total_Sales
Office_Supplies	697286.2596304683
Furniture	721272.9372456992
Technology	835908.0669999998

Insight

Technology category generated the highest total sales, followed by Furniture and Office Supplies. This shows that technology products contributed the most revenue in the dataset.

Region Wise Sales

```
[29]: #Region Wise Sales

df.groupBy("Region") \
  .agg(sum("Sales").alias("Total_Sales")) \
  .show()
```

Region	Total_Sales
South	383004.82276093157
Central	494508.92879999993
East	666958.5020000006
West	710003.0103152348

Insight

The West region recorded the highest sales among all regions, while the South region generated the lowest sales. This indicates that sales performance varies significantly across regions.

Top Products by Sales

```
[31]: #Top 10 Products by Sales

df.groupby("Product Name") \
  .agg(sum("Sales").alias("Total_Sales")) \
  .orderBy(col("Total_Sales").desc()) \
  .show(10, truncate=False)
```

Product Name	Total_Sales
Canon imageCLASS 2200 Advanced Copier	61599.824
Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	27453.384000000002
Cisco TelePresence System EX90 Videoconferencing Unit	22638.48
HON 5400 Series Task Chairs for Big and Tall	21870.576
GBC DocuBind TL300 Electric Binding System	19823.479
GBC Ibimaster 500 Manual ProClick Binding System	19024.5
Hewlett Packard LaserJet 3310 Copier	18839.686
HP DesignJet T520 Inkjet Large Format Printer - 24" Color	18374.895
GBC DocuBind P400 Electric Binding System	17965.068
High Speed Automatic Electric Letter Opener	17030.311999999998

only showing top 10 rows

Insight

A few products contributed a large portion of total sales. Products such as Canon imageCLASS printers and high-end office equipment generated the highest revenue and can be considered key revenue-driving products.

Top Customers by Sales

```
[32]: #Top 10 Customers

df.groupby("Customer Name") \
  .agg(sum("Sales").alias("Total_Sales")) \
  .orderBy(col("Total_Sales").desc()) \
  .show(10, truncate=False)
```

Customer Name	Total_Sales
Sean Miller	25043.05
Tamara Chand	19017.848
Raymond Buch	15117.338999999998
Tom Ashbrook	14595.62
Adrian Barton	14355.610999999999
Sanjit Chand	14129.634000000002
Ken Lonsdale	14071.917
Hunter Lopez	12873.297999999997
Sanjit Engle	12209.438000000002
Christopher Conant	12129.072

only showing top 10 rows

Insight

A small number of customers generated significantly higher sales than others. These customers are important for the business and may be targeted for retention and loyalty programs.

GroupBy and Aggregation Functions


```
[148]: from pyspark.sql.functions import count, sum, avg

df.groupBy("Category").agg(
    count("*").alias("Total_Orders"),
    sum("Sales").alias("Total_Sales"),
    avg("Sales").alias("Average_Sales")
).show()
```

Category	Total_Orders	Total_Sales	Average_Sales
Office Supplies	6026	788107.9504958167	130.78459185128057
Furniture	2121	737828.9746430283	347.8684463192024
Technology	1847	837792.1855596944	453.5962022521356

Insight

Data was grouped by Category using `groupBy()`. Count, sum, and average functions were applied to understand the number of orders, total sales, and average sales for each category.

9. Filtering Operations

Shuffle and Wide Transformation

Shuffle is the process of moving data between different partitions during operations such as `groupBy()`, `join()`, and `reduceByKey()`. Since data needs to be exchanged across partitions, these operations are known as wide transformations. Shuffle can increase execution time because additional data movement is required.

```
[146]: #shuffle
from pyspark.sql.functions import sum

df.groupBy("Category") \
    .agg(sum("Sales").alias("Total_Sales")) \
    .show()
```

Category	Total_Sales
Office Supplies	788107.9504958167
Furniture	737828.9746430283
Technology	837792.1855596944

The `groupBy()` operation is considered a wide transformation because Spark needs to move data across partitions before calculating the aggregation. This data movement process is called Shuffle.

In this assignment, the `groupBy()` operation was used to perform category-wise and region-wise analysis. During these operations, Spark internally performs a shuffle to reorganize data before calculating aggregations. Although the dataset was small, this helped in understanding how Spark processes grouped data efficiently.

Filtering High Sales Records

In this step, I filtered all orders where the Sales value was greater than 1000. This helped me identify high-value sales transactions and see which customers and products generated higher revenue.

```
[33]: #Filtering
      from pyspark.sql.functions import col
      df.filter(col("Sales") > 1000) \
        .select("Customer Name","Category","Sales","Profit") \
        .show(10, truncate=False)
```

Customer Name	Category	Sales	Profit
Christopher Martinez	Office Supplies	6354.95	3177.475
Ken Lonsdale	Technology	1979.928	148.4946
Kristen Hastings	Technology	3499.93	909.9818
Philip Fox	Furniture	1266.86	291.3778
Mitch Willingham	Technology	1919.976	215.9973
Ted Trevino	Furniture	2404.704	150.294
Ralph Arnett	Technology	1199.96	224.9925
Tonja Turnell	Technology	1718.4	150.36
Robert Waldorf	Office Supplies	1924.16	312.676
Cindy Stewart	Technology	4499.985	-6599.978

only showing top 10 rows

Filtering Technology Category Products

In this step, I displayed only the records belonging to the Technology category. This helped me analyze technology products separately and understand their sales performance.

```
[34]: df.filter(col("Category") == "Technology") \
      .select("Product Name","Category","Sales") \
      .show(10, truncate=False)
```

Product Name	Category	Sales
AT&T 1070 Corded Phone	Technology	445.96
AT&T 841000 Phone	Technology	207.0
Microsoft Natural Ergonomic Keyboard 4000	Technology	47.984
Plantronics Audio 995 Wireless Stereo Headset	Technology	263.88
Mitel MIVoice 5330e IP Phone	Technology	1979.928
Memorex Mini Travel Drive 16 GB USB 2.0 Flash Drive	Technology	111.79
Anker 24W Portable Micro USB Car Charger	Technology	35.168
Logitech G500s Laser Gaming Mouse with Adjustable Weight Tuning	Technology	349.95
SanDisk Ultra 32 GB MicroSDHC Class 10 Memory Card	Technology	66.3
LG Exalt	Technology	280.782

only showing top 10 rows

Identifying Loss-Making Products

In this step, I filtered all records where Profit was less than 0. These records represent products that resulted in a loss. This analysis helps identify products that may need attention from the business.

```
[113]: df.filter(col("Profit") < 0) \
        .select("Product Name","Sales","Profit") \
        .show(10, truncate=False)
```

Product Name	Sales	Profit
Bretford CR4500 Series Slim Rectangular Table	NULL	-383.031
Holmes Replacement Filter for HEPA Air Cleaner, Very Large Room, HEPA Filter	68.81	-123.858
Storex DuraTech Recycled Plastic Frosted Binders	12.544	-3.816
Global Deluxe Stacking Chair, Gray	71.372	-1.0196
Riverside Palais Royal Lawyers Bookcase, Royale Cherry Finish	3083.43	-1665.0522
Avery Recycled Flexi-View Covers for Binding Systems	9.618	-7.0532
Electric Architect's Clamp-On Swing Arm Lamp, Black	190.92	-147.963
Atlantic Metals Mobile 3-Shelf Bookcases, Custom Colors	532.3992	-46.9764
Global Fabric Manager's Chair, Dark Gray	212.058	-15.147
Global Value Mid-Back Manager's Chair, Gray	213.115	-15.2225

only showing top 10 rows

Group By and aggregation

Category-wise aggregation showed the overall sales performance of each product category and helped identify high-performing categories.

```
[30]: #Group By + Revenue aggregation

from pyspark.sql.functions import sum, avg, max, min

df.groupBy("Category").agg(
    sum("Sales").alias("Total_Sales"),
    avg("Sales").alias("Average_Sales"),
    max("Sales").alias("Max_Sales"),
    min("Sales").alias("Min_Sales")
).show()
```

Category	Total_Sales	Average_Sales	Max_Sales	Min_Sales
Office Supplies	697286.2596304683	123.588489831703	9892.74	0.444
Furniture	721272.9372456992	351.6689113825935	4416.174	1.892
Technology	835900.0669999998	454.5405475802065	22638.48	0.99

Creating a New Column (Profit_Status)

I created a new column called Profit_Status using the Profit column. If the profit value was greater than 0, the record was marked as "Profit"; otherwise, it was marked as "Loss". This made it easier to classify transactions based on their profitability.

```
[91]: #creating a new column

from pyspark.sql.functions import when

df = df.withColumn(
    "Profit_Status",
    when(col("Profit") > 0, "Profit")
    .otherwise("Loss")
)

df.select("Profit","Profit_Status").show(10)
```

Profit	Profit_Status
12.0276	Profit
23.3928	Profit
3177.475	Profit
55.745	Profit
10.3152	Profit
51.75	Profit
0.5998	Loss
1.2792	Profit
8.0997	Profit
4.4892	Profit

only showing top 10 rows

10.Data Processing Pipeline

The complete workflow followed in this assignment was:

[illegible]

```
[149]: pipeline_df = (
    df
    .dropDuplicates()
    .na.fill({"Sales": 0})
)

pipeline_df.groupBy("Category") \
    .agg(sum("Sales").alias("Total_Sales")) \
    .show()
```

Category	Total_Sales
Office Supplies	788107.9504958167
Furniture	737828.9746430283
Technology	837792.1855996944

This pipeline ensured that the dataset was cleaned, transformed, and analyzed efficiently using PySpark.

11. Conclusion

In this assignment, Apache Spark and PySpark DataFrames were used to perform data cleaning, transformation, filtering, and aggregation operations on the Superstore dataset.

Null values and duplicate records were successfully identified and handled. Data types were converted to appropriate formats for analysis. Various filtering and aggregation operations were performed to generate useful business insights.

This assignment helped in understanding Spark DataFrame operations and building a complete data processing workflow.